

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

## ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

|                  |                                                                                                                                                          |               |                                                 |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-------------------------------------------------|
| Course Code:     | DSA03                                                                                                                                                    | Course Title: | Natural Language Processing                     |
| CO Assessed:     | CO1 – Imitation (L1)                                                                                                                                     | Assessment:   | Assessment Tool 1 – Application Based Questions |
| Weightage:       | 40% of CIA                                                                                                                                               |               |                                                 |
| CO1 Statement:   | Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3) |               |                                                 |
| Student Name:    | S.Krishna Chaithanya                                                                                                                                     | Reg. No.:     | 192424287                                       |
| Section / Batch: | B.Tech-AIDS/2024                                                                                                                                         | Date:         | 16/07/2026                                      |

### Code of Conduct

*I S.Krishna Chaithanya (192424287) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: S.Krishna Chaithanya

### Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

| Section / Topic             | Focus                              | Q Nos |
|-----------------------------|------------------------------------|-------|
| Information Extraction      | Regex-Based Information Extraction | 1     |
| Pattern Matching            | Regex Pattern Matching             | 2     |
| Regular Expression Patterns | Regex-Based Input Validation       | 3     |

### Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

**Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:**

Extract the candidate's name.  
Identify email addresses.  
Extract mobile numbers.  
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).  
Extract years of experience.  
Generate a structured summary of the candidate's profile.  
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

### Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

**Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:**

Search products using an exact keyword.  
Perform prefix-based searches.  
Perform suffix-based searches.  
Search using partial keywords.  
Perform case-insensitive searches.  
Display all matching product names.  
Generate a report showing the total number of matching products for each search.

### Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

**Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:**

Validate the student register number.  
Validate the institutional email address.  
Validate the course code.  
Validate the semester information.  
Validate the student's mobile number.  
Display appropriate validation messages for each field.  
Generate a final registration status report indicating whether the registration is successful.

# 1.Resume Information Extraction using Regular Expressions

## Aim

Develop a Python program using Regular Expressions (Regex) to extract information from resumes and identify eligible candidates.

## Algorithm

1. Import the re module.
2. Read the resume text.
3. Extract:
  - Candidate Name
  - Email Address
  - Mobile Number
  - Technical Skills
  - Years of Experience
4. Generate a structured summary.
5. Check eligibility:
  - Experience  $\geq$  2 years
  - Python skill available
6. Display eligibility status.

## Python Program

```
import re
resume = """
Name: Rahul Sharma
Email: rahul.sharma@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 years
"""

name = re.search(r"Name:\s*(.*)", resume)
name = name.group(1) if name else "Not Found"

email = re.search(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)
email = email.group() if email else "Not Found"

mobile = re.search(r"\b\d{10}\b", resume)
mobile = mobile.group() if mobile else "Not Found"

skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
found_skills = []
for skill in skills:
    if re.search(skill, resume, re.IGNORECASE):
        found_skills.append(skill)

exp = re.search(r"(\d+)\s+years", resume)
experience = int(exp.group(1)) if exp else 0
```

```

print("Candidate Summary")
print("-----")
print("Name :", name)
print("Email :", email)
print("Mobile :", mobile)
print("Skills :", " ", ".join(found_skills))
print("Experience :", experience, "Years")
if experience >= 2 and "Python" in found_skills:
    print("Status : Eligible")
else:
    print("Status : Not Eligible")

```

## Regular Expressions Used

| Purpose    | Regex                                          |
|------------|------------------------------------------------|
| Name       | Name:\s*(.*)                                   |
| Email      | [A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,} |
| Mobile     | \b\d{10}\b                                     |
| Experience | (\d+)\s+years                                  |

## Sample Input

Name: Rahul Sharma  
 Email: rahul.sharma@gmail.com  
 Mobile: 9876543210  
 Skills: Python, Java, SQL, Machine Learning, NLP  
 Experience: 3 years

## Sample Output

Candidate Summary  
 -----  
 Name : Rahul Sharma  
 Email : rahul.sharma@gmail.com  
 Mobile : 9876543210  
 Skills : Python, Java, SQL, Machine Learning, NLP  
 Experience : 3 Years  
 Status : Eligible

## Test Cases

| Experience | Python Skill | Result       |
|------------|--------------|--------------|
| 3          | Yes          | Eligible     |
| 5          | No           | Not Eligible |
| 1          | Yes          | Not Eligible |
| 0          | No           | Not Eligible |

## Conclusion

The program successfully extracts resume information using Regular Expressions and identifies eligible candidates based on the required criteria.

### 1. Product Search System using Regular

**Expressions Aim:** Develop a Python product search system using Regex. **Algorithm**

1. Store product names in a list.
2. Accept a search keyword.
3. Perform:
  - Exact search
  - Prefix search
  - Suffix search
  - Partial search
  - Case-insensitive search
4. Display matching products.
5. Display total matches.

## Python

### Program

```
import re
products = [
    "Laptop",
    "Laptop Bag",
    "Wireless Mouse",
    "Gaming Keyboard",
    "USB Cable",
    "Smart Watch",
    "Phone Charger",
    "Bluetooth Speaker"
]
keyword = input("Enter Search Keyword: ")
exact = []
prefix = []
suffix = []
partial = []
ignore = []

for p in products:
    if re.fullmatch(keyword, p):
        exact.append(p)
    if re.match(keyword, p):
        prefix.append(p)
    if re.search(keyword + r"$", p):
        suffix.append(p)
    if re.search(keyword, p):
        partial.append(p)
```

```

    if re.search(keyword, p, re.IGNORECASE):
        ignore.append(p)
print("\nExact Search :", exact)
print("Prefix Search :", prefix)
print("Suffix Search :", suffix)
print("Partial Search :", partial)
print("Case Insensitive :", ignore)
print("\nReport")
print("Exact :", len(exact))
print("Prefix :", len(prefix))
print("Suffix :", len(suffix))
print("Partial :", len(partial))
print("Case Insensitive :", len(ignore))

```

## Sample Input

Laptop

## Sample Output

Exact Search : ['Laptop']  
 Prefix Search : ['Laptop', 'Laptop Bag']  
 Suffix Search : ['Laptop']  
 Partial Search : ['Laptop', 'Laptop Bag']  
 Case Insensitive : ['Laptop', 'Laptop Bag']

## Report

Exact : 1  
 Prefix : 2  
 Suffix : 1  
 Partial : 2  
 Case Insensitive : 2

## Regular Expressions

| Search      | Regex          |
|-------------|----------------|
| Exact       | re.fullmatch() |
| Prefix      | re.match()     |
| Suffix      | keyword\$      |
| Partial     | re.search()    |
| Ignore Case | re.IGNORECASE  |

## Conclusion

The program supports multiple Regex-based search methods and generates a report of matching products.

## 2. University Registration Validation using Regular Expressions 4.

**Aim:** Develop a Python validation system for university registration.

### Algorithm

1. Read student details.
2. Validate:
  - Register Number
  - Institutional Email
  - Course Code
  - Semester
  - Mobile Number
3. Display validation messages.
4. Generate final registration status.

### Python Program

```
import re
reg = input("Register Number: ")
email = input("Institution Email: ")
course = input("Course Code: ")
semester = input("Semester: ")
mobile = input("Mobile Number: ")
reg_valid = re.fullmatch(r"\d{2}[A-Z]{3}\d{4}", reg)
email_valid = re.fullmatch(r"[A-Za-z0-9._%+-]+@saveetha\.com", email)
course_valid = re.fullmatch(r"[A-Z]{3}\d{3}", course)
semester_valid = re.fullmatch(r"[1-8]", semester)
mobile_valid = re.fullmatch(r"[6-9]\d{9}", mobile)
print("\nValidation Report")
print("Register Number :", "Valid" if reg_valid else "Invalid")
print("Email :", "Valid" if email_valid else "Invalid")
print("Course Code :", "Valid" if course_valid else "Invalid")
print("Semester :", "Valid" if semester_valid else "Invalid")
print("Mobile :", "Valid" if mobile_valid else "Invalid")
if reg_valid and email_valid and course_valid and semester_valid and mobile_valid:
    print("\nRegistration Successful")
else:
    print("\nRegistration Failed")
```

### Regular Expressions

| Field           | Regex                           |
|-----------------|---------------------------------|
| Register Number | \d{2}[A-Z]{3}\d{4}              |
| Email           | [A-Za-z0-9._%+-]+@saveetha\.com |
| Course Code     | [A-Z]{3}\d{3}                   |
| Semester        | [1-8]                           |
| Mobile          | [6-9]\d{9}                      |

## Sample Input

Register Number  
: 23AID1024  
Email :  
student@saveetha.com  
Course Code : NLP301  
Semester : 5  
Mobile : 9876543210

## Sample

## Output

Validation

Report

Register

Number :

Valid

Email :

Valid

Course

Code :

Valid

Semester

: Valid

Mobile :

Valid

Registration Successful

## Test Cases

| Register No | Email   | Course  | Semester | Mobile | Status     |
|-------------|---------|---------|----------|--------|------------|
| Valid       | Valid   | Valid   | Valid    | Valid  | Successful |
| Invalid     | Valid   | Valid   | Valid    | Valid  | Failed     |
| Valid       | Invalid | Valid   | Valid    | Valid  | Failed     |
| Valid       | Valid   | Invalid | Valid    | Valid  | Failed     |

## Conclusion

The program validates all registration fields using Regular Expressions and generates a final registration status. These three solutions cover all the requirements in your assessment and are structured to align with the rubric for



### Rubrics for Calculation

| Criteria                                | Weightage | Level 4 – Exemplary                                                        | Level 3 – Proficient                                  | Level 2 – Developing                           | Level 1 – Beginning                                    |
|-----------------------------------------|-----------|----------------------------------------------------------------------------|-------------------------------------------------------|------------------------------------------------|--------------------------------------------------------|
| <b>Understanding of Problem Context</b> | 20        | Clearly understands the problem and accurately identifies all requirements | Good understanding with minor gaps                    | Basic understanding; some requirements unclear | Poor understanding or misinterpretation of the problem |
| <b>Application of Concepts</b>          | 30        | Accurately applies appropriate concepts to solve complex problems          | Applies relevant concepts correctly with minor errors | Applies basic concepts with limited accuracy   | Fails to apply appropriate concepts                    |
| <b>Problem-Solving Approach</b>         | 20        | Logical, well-structured, and efficient approach                           | Mostly logical approach with minor gaps               | Basic approach with some inconsistencies       | Illogical or unclear approach                          |
| <b>Accuracy of Solution</b>             | 20        | Completely correct solution with proper justification                      | Mostly correct with minor errors                      | Partially correct solution                     | Incorrect or incomplete solution                       |
| <b>Clarity</b>                          | 10        | Clear, well-organized, and properly explained solution                     | Understandable with minor clarity issues              | Limited clarity and explanation                | Poorly presented and difficult to understand           |

| Q. No. / Criteria | Understanding of Problem Context | Application of Concepts | Problem-Solving Approach | Accuracy of Solution | Clarity | Sub. Total | Total % |
|-------------------|----------------------------------|-------------------------|--------------------------|----------------------|---------|------------|---------|
| 1                 |                                  |                         |                          |                      |         |            |         |
| 2                 |                                  |                         |                          |                      |         |            |         |
| 3                 |                                  |                         |                          |                      |         |            |         |

| Faculty In-charge | Course Coordinator | Program Director |
|-------------------|--------------------|------------------|
|                   |                    |                  |
| Signature: _____  | Signature: _____   | Signature: _____ |
| Date: _____       | Date: _____        | Date: _____      |